Ellis Communication Kit

One system, one set of facts, four views. How to explain Ellis to the person accountable for it, to governance, to the platform's developers and to whoever runs it, without the story changing between them.

As of 5 Nov 2026 **Prepared by** Quincy Nkwain Ninying, Forward Deployed Engineer **Accountable** K.D. **Governance** A.M.

FIXED FACTS

Identical in every view below

These do not change with the audience. If a view ever implies something different, the view is wrong, not the fact.

What Ellis is	**The platform's reliability agent, built in phases. This phase is its evidence and judgement layer, with no model calls at runtime.**	Verified by the platform	**39 of 48 capabilities; 46 of the plan's 55 stories**
Sandbox rungs	**16 of 16 built (R3 and R4); 17 shadow-mode rungs wait on K.D.'s thresholds**	Writes to the platform	**None. Merging and deploying are default-denied to agents; a person does both.**
Findings	**82; of the 76 A.M. reviewed, 61 held as written or worse; 14 fixed upstream**	Severity	**Set only by K.D.'s table; until it has rules, every severity is unassigned**
Tests	**883, each key rule broken on purpose and caught**	Core limitation	**Ellis sees only what it is given: the platform's source and exports a person supplies. An answer is about that input, at that time.**

01 · accountable FOR K.D.

Set the rules that let Ellis go further

The decisions on the table: **the severity table and the shadow-mode thresholds**, starting with the five first monitors, and signing rungs R0 to R2 as the evidence arrives.

WHAT YOU GET TODAY

Answers to your reliability questions, each with the rows behind it and what it could not see, and a briefing you can read with no login, the unmeasured part first. Findings the platform has acted on: 14 fixed upstream.

WHAT IT ASKS OF YOU

Decisions, not time: ten are open, and three unlock the most. And an export now and then, because Ellis has no production access: a person with admin access pulls the records it reads.

RISK, IN ONE LINE

Ellis cannot change the platform, so its risk is a wrong or incomplete answer. Two rules contain it: a check that cannot see reads as unknown, never as clean, and every answer names its blind spots.

HONEST LIMITATION

An answer is only as current as the export behind it. Ellis does not watch the platform live today; that comes with the entry points and the gate, which are not built yet.

DECISION REQUESTED

Set the severity table and the shadow-mode thresholds for the five first monitors.

02 · governance FOR A.M.

Name who builds the control boundary

The decisions on the table: **who builds the gate, tool broker, agent core, entry points and audit log**, and where they run; republishing the plan with rung ids; and signing rungs R3 and up.

WHAT HAS BEEN GOVERNED

Every change through the same review: 20 working rules adopted from named decisions, 76 findings reviewed line by line, every capability verified from a commit that names its story, and every sandbox rung run beside a control that must fail.

THE BOUNDARIES IN CODE TODAY

Every check reads and none writes. A problem or an ok with no row behind it cannot leave a tool. Rehearsals run only on sandbox copies, and the R3 ones only when a named person starts them.

WHAT CHANGES WITH THE GATE AND BROKER

Today a person runs each command. With the gate and broker, cases start on their own and the checks become the agent's tools, so the boundaries move from "a person ran it" to "the gate allowed it", and the audit log records both.

HONEST LIMITATION

The controls are proven in code and tests, not yet enforced at runtime. Until the gate exists, nothing stops a person from running a command outside the process; the process is what holds.

DECISION REQUESTED

Name one owner and a host for the gate and broker, and republish the plan with rung ids.

03 · platform developers FOR THE PEOPLE WHO OWN THE CODE A FINDING POINTS AT

Act on a finding, or show it is wrong

The decision on the table: **fix what a finding describes, or contest it** with the commit that shows otherwise.

WHAT A FINDING GIVES YOU

The file and line, at the commit it was read at; the evidence; and where one exists, a sandbox reproduction or a baseline test that fails for the reported reason, so the fix can be judged against it.

HOW TO CONTEST ONE

Point at the commit. Ellis re-reads the code and corrects its answers in the open: when half of finding 54 was fixed on 28 October, both capabilities that described it were corrected the day the fix was found on main.

WHAT ELLIS NEVER DOES TO YOUR CODE

It opens no pull request, edits no file and changes no row. A fix ships through your reviewed deploy, by a person. Three open findings matter most: critical security alerts never become tickets (8), lead-drop tickets close as "cleared" on a failed query (20), and the intelligence cleanup can delete rows it never archived (9).

HONEST LIMITATION

Citations are pinned to a commit, and the code moves. Check a finding against main before acting on it; if it has moved, say so and Ellis re-reads it.

ASK

Take findings 8, 20 and 9 first; reply with a commit when one is fixed.

04 · operator FOR WHOEVER RUNS THE CHECKS

Trust a verdict, or know exactly why not

The decision on the table, every time you run a check: **is this answer good enough to act on**, or does it need a better input first?

WHAT YOU RUN

40 commands, organised in the Playbook by the question they answer, and 16 sandbox rehearsals. Each says what it needs and refuses with exit 1 if something is missing.

WHAT YOU READ

The verdict first, then the rows it rests on, then the blind spots. Ok means "true of these rows at this time"; unknown means "could not look", never "fine".

WHEN IT SAYS UNKNOWN

Fix the input, not the answer: re-export, or re-clone. If the record does not exist on the platform at all, that absence is itself the finding, and it goes to K.D.

HONEST LIMITATION

On the command line, each check answers in its own words; the shared verdicts are the envelope the broker will use. The Playbook maps one to the other, and reading the blind spots is on you.

ASK

Start with the reliability briefing, and follow the Playbook's step for each verdict.

WORDS

What to say, and what never to say

The same words in every view keep the facts above from drifting.

SAY	DO NOT SAY	WHY
An agent built in phases; this phase has no model calls at runtime	"Not an agent", "just monitoring"	The model step comes later, under the same rules; the current phase is its evidence layer.
Verified by the platform	"Verified" for work that is submitted or confirmed from the repo	Only the platform's verification state counts; submitted is awaiting review.
Sandbox only; nothing ran on the platform	"Tested in production", "fixed"	Every rehearsal runs on copies, and a fix is a person's reviewed deploy.
Unknown: it could not look	"No problems found" when a check could not read	Unknown is decided before clean, by design.
Ellis reports; K.D. decides	"Ellis fixed", "Ellis approved"	Ellis changes nothing and decides nothing; people do both.
Ellis's output shows	"I found a bug in your code"	Defects are Ellis's to find and report through its output; the findings register closed at 82.
Severity unassigned, until K.D.'s table has a rule	"Critical" or "low" from a feeling	Severity is K.D.'s decision, applied by code.

Sources: the capability catalog and README; the platform's progress file (46 of 55 stories verified, 5 Nov 2026); the Colaberry Platform Reliability Audit; the output contract (`src/output/toolOutput.ts`, decision record 006); K.D.'s architecture of 28 Oct 2026. Companion documents: Ellis Executive Brief, Ellis Playbook, Ellis Workflow.